Data Structures & Algorithms

Exam Study Guide

Big O | QuickSort | HeapSort | Stacks | Queues | Trees | BST

0. Big O Notation

Big O notation describes how the runtime or space requirements of an algorithm grow as the input size (n) grows. It gives the WORST-CASE upper bound — we ignore constants and lower-order terms.

Rule of thumb: ask yourself 'if n doubles, what happens to the time?'

Notation	Name	Example	Performance
`O(1)`	Constant	Array index access	BEST — doesn't grow with n
`O(log n)`	Logarithmic	Binary Search	Excellent — halves problem each step
`O(n)`	Linear	Linear Search, loop once	Good — grows proportionally
`O(n log n)`	Linearithmic	QuickSort avg, MergeSort	Good — most efficient sorts
`O(n²)`	Quadratic	Bubble Sort, nested loops	Slow — avoid for large n
`O(2ⁿ)`	Exponential	Recursive Fibonacci (naive)	WORST — avoid entirely

0.1 How to Identify Big O

- Single loop over n elements → O(n)
- Loop inside a loop (both over n) → O(n²)
- Halving the problem each step (like Binary Search) → O(log n)
- Sorting-then-looping → O(n log n)
- Constant-time operation (no loop, no recursion) → O(1)

0.2 Big O of Common Data Structure Operations

Structure	Access	Search	Insert	Delete
Array	`O(1)`	`O(n)`	`O(n)`	`O(n)`
Stack	`O(n)`	`O(n)`	`O(1)`	`O(1)`
Queue	`O(n)`	`O(n)`	`O(1)`	`O(1)`

Binary Search	`O(log n)`	`O(log n)`	—	—
BST (avg)	`O(log n)`	`O(log n)`	`O(log n)`	`O(log n)`
Min-Heap	`O(1) min`	`O(n)`	`O(log n)`	`O(log n)`

1. QuickSort Algorithm

QuickSort is a divide-and-conquer sorting algorithm. It selects a PIVOT element and partitions the array around it, then recursively sorts each sub-array.

1.1 How It Works

- Choose a pivot (usually the last element)
- Partition: move all elements <= pivot to the LEFT, all > pivot to the RIGHT
- Recursively sort the left and right sub-arrays
- Base case: array of size 0 or 1 is already sorted

1.2 Partition Algorithm (Java)

```
public static int partition(int[] arr, int low, int high) {
    int pivot = arr[high];       // last element is pivot
    int i = (low - 1);           // i = boundary of smaller elements

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {   // current element <= pivot?
            i++;                 // expand 'smaller' region
            int temp = arr[i];   // swap arr[i] and arr[j]
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    // Place pivot in its correct final position
    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    return i + 1;                // return pivot's final index
}
```

1.3 QuickSort Recursive Method (Java)

```
public static void quickSort(int[] arr, int low, int high) {
    if (low < high) {
        int pivotIndex = partition(arr, low, high);
        quickSort(arr, low, pivotIndex - 1);   // sort left sub-array
        quickSort(arr, pivotIndex + 1, high);  // sort right sub-array
    }
}
```

1.4 Full Partition Trace — [90, 23, 5, 109, 12]

STEP 1 — Partition [90, 23, 5, 109, 12]: pivot = 12, i = -1

- j=0: 90 > 12 → skip
- j=1: 23 > 12 → skip
- j=2: 5 <= 12 → i++ (i=0), swap [0]↔[2] → [5, 23, 90, 109, 12]

- j=3: 109 > 12 → skip
- Place pivot: swap [i+1=1]↔[last=4] → [5, 12, 90, 109, 23]

Result: Pivot 12 at index 1 — FINAL POSITION ✓ | Left sub-array [5] is single element

STEP 2 — Partition [90, 109, 23]: pivot = 23, i = -1

- j=0: 90 > 23 → skip
- j=1: 109 > 23 → skip
- Place pivot: swap [i+1=0]↔[last=2] → [23, 109, 90]

Result: Pivot 23 at index 0 — FINAL POSITION ✓

STEP 3 — Partition [109, 90]: pivot = 90, i = -1

- j=0: 109 > 90 → skip
- Place pivot: swap [i+1=0]↔[last=1] → [90, 109]

Result: Pivot 90 at index 0 — FINAL POSITION ✓

FINAL SORTED ARRAY: [5, 12, 23, 90, 109] ✓

1.5 Big O of QuickSort

Best Case	O(n log n) — pivot divides array evenly each time
Average Case	O(n log n)
Worst Case	O(n^2) — already sorted array with last-element pivot
Space	O(log n) — recursive call stack depth

---

2. Heap Sort

Heap Sort uses a Max-Heap (or Min-Heap) to sort an array. It first builds a heap from the array, then repeatedly extracts the root (max/min) and places it at the end.

2.1 How Heap Sort Works

- Phase 1 — Build a Max-Heap from the unsorted array (heapify)
- Phase 2 — Repeatedly: swap the root (largest) with the last element, reduce heap size by 1, then heapify the root down
- Result: elements are extracted in descending order → array is sorted ascending

2.2 Heap Sort Trace — [109, 90, 23, 12, 5]

Starting with Max-Heap already built: root = 109 (largest)

Step 1 — Extract 109: Swap root (109) with last element (5)
- Array view: [5, 90, 23, 12 | 109] (109 is locked/sorted)
- Heapify: 5 < child 90 → swap → root becomes 90
- Array view: [90, 5, 23, 12 | 109]
- Continue heapify: 5 < child 12 → swap
- Array view: [90, 12, 23, 5 | 109]

Step 2 — Extract 90: Swap root (90) with last active element (5)
- Array view: [5, 12, 23 | 90, 109] (90 is locked)
- Heapify: 5 < child 23 → swap → root becomes 23
- Array view: [23, 12, 5 | 90, 109]

Step 3 — Extract 23: Swap root (23) with last active element (5)
- Array view: [5, 12 | 23, 90, 109] (23 is locked)
- Heapify: new root is 5. Swap with bigger child 12
- Array view: [12, 5 | 23, 90, 109]

Step 4 — Extract 12: Swap root (12) with last active element (5)
- Array view: [5 | 12, 23, 90, 109] (12 is locked)
- Only 1 element left (5) — we are done!

FINAL SORTED ARRAY: [5, 12, 23, 90, 109] ✓

2.3 Big O of Heap Sort

Best Case	O(n log n)
Average Case	O(n log n)
Worst Case	O(n log n) — always consistent!
Space	O(1) — sorts in-place (unlike MergeSort)

3. Stack Operations & Recursion

3.1 What is a Stack?

A stack is a linear data structure that follows the LIFO (Last In, First Out) principle. Think of a stack of plates — you can only add or remove from the top.

Visual representation:

```
TOP → | Item 4 |  ← push/pop here
      | Item 3 |
      | Item 2 |
      | Item 1 |
```

3.2 Key Stack Operations

LIFO	Last In, First Out — the defining property of a stack
push()	Add an element to the TOP of the stack
pop()	Remove and return the element from the TOP
peek()	View the top element WITHOUT removing it
isEmpty()	Check if the stack has no elements (returns boolean)
isFull()	Check if the stack is full (array-based stacks only)

3.3 How Recursion Uses the Call Stack

Every recursive method call is PUSHED onto the call stack. When the BASE CASE is reached, calls are POPPED off (unwound) in REVERSE ORDER.

Example: factorial(4) — trace the call stack:

```
factorial(4) → PUSHED onto stack
  factorial(3) → PUSHED
    factorial(2) → PUSHED
      factorial(1) → BASE CASE (returns 1)

      factorial(1) → POPPED: returns 1
    factorial(2) → POPPED: 2 x 1 = 2
  factorial(3) → POPPED: 3 x 2 = 6
factorial(4) → POPPED: 4 x 6 = 24  ← final answer
```

Key insight: Recursion pushes n frames onto the stack before unwinding. This is why deep recursion can cause StackOverflowError!

3.4 Stack Applications

- Expression evaluation — e.g. postfix/prefix calculations

- Undo/redo operations in editors
- Browser back/forward navigation
- Balancing parentheses/brackets
- Depth-First Search (DFS) in graphs

---

4. Queue Operations & Circular Queue

4.1 What is a Queue?

A queue is a linear data structure that follows the FIFO (First In, First Out) principle. Like a queue at a shop — the first person to join is the first to be served.

```
FRONT → [A] → [B] → [C] → [D] ← REAR
  dequeue ←                  → enqueue
```

4.2 Queue Operations

enqueue()	Add element x at the REAR of the queue
dequeue()	Remove and return element from the FRONT
peek()	View the FRONT element without removing it
isEmpty()	Returns true if queue has no elements
isFull()	Returns true if queue is at capacity (array-based)

4.3 Problems with Linear Queue

In a simple array-based queue, once elements are dequeued, the FRONT pointer moves forward. This WASTES MEMORY because empty slots at the front cannot be reused.

```
Index:  0     1     2     3     4
      [--]  [--]  [C]   [D]   [E]
                  ^             ^
                FRONT          REAR
```

Indices 0 and 1 are WASTED — they cannot be reused in a linear queue.

4.4 Circular Queue

A circular queue solves the wasted space problem. When the REAR pointer reaches the end of the array, it wraps around to index 0 using the modulo operator.

Wrap formula: rear = (rear + 1) % CAPACITY
Front formula: front = (front + 1) % CAPACITY

After wrapping — a new element is enqueued at index 0:

```
Index:  0     1     2     3     4
      [F]   [--]  [C]   [D]   [E]
       ^           ^
    REAR=0      FRONT=2
```

FRONT = 2 (C), REAR = 0 — the queue has wrapped around successfully.

4.5 Circular Queue Java Code — enqueue & dequeue

```
void enqueue(int val) {
    if (isFull()) throw new RuntimeException("Full");
    rear = (rear + 1) % CAPACITY;
    arr[rear] = val;
    if (front == -1) front = rear;  // first element
}
```

```
int dequeue() {
    if (isEmpty()) throw new RuntimeException("Empty");
    int value = arr[front];
    if (front == rear) { front = -1; rear = -1; }  // last element
removed
    else front = (front + 1) % CAPACITY;
    return value;
}
```

4.6 isEmpty and peek

```
boolean isEmpty() {
    return front == -1 && rear == -1;
}
```

```
int peek() {
    if (!isEmpty()) return arr[front];
    else throw new RuntimeException("Queue is empty");
}
```

4.7 Priority Queue using Min-Heap

A Priority Queue processes elements in order of priority, not arrival. The most efficient implementation uses a Min-Heap — a Binary Heap where the MINIMUM value always stays at the root.

Min-Heap tree example:

```
          [1]
         /   \
       [4]   [3]
      /  \   /  \
    [9] [7][8] [10]
```

- dequeue() always removes the ROOT (smallest = highest priority)
- After removal, the last element is moved to root, then heapify-down restores the heap
- enqueue() adds at the end, then heapify-up to restore heap property
- Time complexity: O(log n) for both enqueue and dequeue

5. Binary Trees & Tree Traversals

5.1 Binary Tree Terminology

Root	The TOP node of the tree — has no parent
Leaf	A node with NO children
Internal Node	A node that has at least one child
Parent	A node directly ABOVE another node
Child	A node directly BELOW its parent
Height	Longest path from root to a leaf (counted in edges)
Depth	Distance (edges) from root to a specific node
Subtree	A node and all its descendants
BST	Binary Search Tree — left < node < right property

Example binary tree:

```
          [50]          ← Root (depth=0)
         /    \
      [30]    [70]      ← Internal nodes (depth=1)
     /   \   /   \
  [20] [40][60]  [80]   ← Leaf nodes (depth=2)
```

Height of tree: = 2 (root is at level 0)
Depth of node 40: = 2
Internal nodes: 50, 30, 70 (have children)
Leaf nodes: 20, 40, 60, 80 (no children)

5.2 Binary Search Tree (BST) Property

A BST is a Binary Tree with a strict ordering rule for EVERY node:

- All values in the LEFT subtree are LESS THAN the current node
- All values in the RIGHT subtree are GREATER THAN the current node
- Both left and right subtrees are ALSO BSTs (recursive property)
- Shorthand: left < root < right

Inserting 50, 30, 70, 20, 40, 60, 80 into a BST:

```
            50
           /  \
         30    70
        /  \   /  \
      20   40 60   80
```

Search for 40: 40 < 50 → go LEFT → 40 > 30 → go RIGHT → FOUND!
Binary search in a BST is O(log n) average — you eliminate half the tree each step.

5.3 Tree Traversals

Tree traversal means visiting every node EXACTLY ONCE. There are 3 Depth-First (DFS) traversals and 1 Breadth-First (BFS) traversal.

Reference tree for all examples below:

```
          [A]
         /   \
       [B]   [C]
       /       \
     [D]       [E]
```

In-Order L → Root → R	Visit left → root → right. Result: D, B, A, C, E Key: In-Order on a BST gives SORTED output!
Pre-Order Root → L → R	Visit root → left → right. Result: A, B, D, C, E Use: Copying or serializing a tree
Post-Order L → R → Root	Visit left → right → root. Result: D, B, E, C, A Use: Deleting or freeing a tree (children first)
BFS / Level-Order	Visit all nodes level by level (uses a Queue internally). Result: A, B, C, D, E Use: Shortest path, level-by-level processing

---

6. Insertion Sort & Selection Sort

6.1 Insertion Sort

Insertion Sort builds the sorted array one element at a time. It takes each element and inserts it into its correct position among the already-sorted elements to its left — like sorting playing cards in your hand.

- Start from index 1 (first element is trivially sorted)
- Pick the current element as the 'key'
- Shift all larger elements to the RIGHT to make space
- Insert the key into the correct position
- Repeat until the whole array is sorted

6.2 Insertion Sort — Java Code

```
public static void insertionSort(int[] arr) {
    int n = arr.length;
    for (int i = 1; i < n; i++) {
        int key = arr[i];          // element to be inserted
        int j = i - 1;             // start comparing left
```

```
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j]; // shift element right
            j--;
        }
        arr[j + 1] = key;          // insert key in correct spot
    }
}
```

6.3 Insertion Sort Trace — [5, 3, 1, 4, 2]

Pass 1 (i=1): key=3 | 5>3 → shift right → [3, 5, 1, 4, 2]
Pass 2 (i=2): key=1 | 5>1 shift, 3>1 shift → [1, 3, 5, 4, 2]
Pass 3 (i=3): key=4 | 5>4 shift, 3<4 stop → [1, 3, 4, 5, 2]
Pass 4 (i=4): key=2 | 5,4,3>2 shift, 1<2 stop → [1, 2, 3, 4, 5]
FINAL: [1, 2, 3, 4, 5] ✓

6.4 Insertion Sort — Big O

Best Case	O(n) — already sorted array (inner while loop never runs)
Average Case	O(n²)
Worst Case	O(n²) — reverse sorted array (maximum shifts every pass)
Space	O(1) — in-place, no extra array needed
Stable?	YES — equal elements keep their original relative order

---

6.5 Selection Sort

Selection Sort divides the array into a SORTED left part and an UNSORTED right part. On each pass it SELECTS the MINIMUM element from the unsorted part and swaps it to the front.

- On each pass, find the minimum element in the unsorted portion
- Swap it with the first element of the unsorted portion
- The sorted boundary moves one step right
- Repeat for n-1 passes

6.6 Selection Sort — Java Code

```
public static void selectionSort(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;              // assume current pos is min
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIdx]) {
                minIdx = j;          // found a new minimum
```

```
            }
        }
        // Swap minimum into sorted position
        int temp = arr[minIdx];
        arr[minIdx] = arr[i];
        arr[i] = temp;
    }
}
```

6.7 Selection Sort Trace — [5, 3, 1, 4, 2]

Pass 1 (i=0): min=1 at index 2 → swap with index 0 → [1, 3, 5, 4, 2]
Pass 2 (i=1): min=2 at index 4 → swap with index 1 → [1, 2, 5, 4, 3]
Pass 3 (i=2): min=3 at index 4 → swap with index 2 → [1, 2, 3, 4, 5]
Pass 4 (i=3): min=4 at index 3 → already in place → [1, 2, 3, 4, 5]
FINAL: [1, 2, 3, 4, 5] ✓

6.8 Selection Sort — Big O

Best Case	O(n²) — always scans the full unsorted portion
Average Case	O(n²)
Worst Case	O(n²) — always the same regardless of input
Space	O(1) — in-place
Stable?	NO — swapping can change relative order of equal elements
vs Insertion	Selection does fewer SWAPS (at most n-1), but more COMPARISONS

6.9 Insertion vs Selection — Side by Side

Property	Insertion Sort	Selection Sort
Best Case	O(n) ← BETTER	O(n²)
Worst Case	O(n²)	O(n²)
Swaps	O(n²) — many shifts	O(n) — at most n-1 swaps
Stable	YES ✓	NO ✗
Best for	Nearly sorted data	Minimising swaps is critical

7. Linear Search & Binary Search

7.1 Linear Search

Linear Search checks each element ONE BY ONE from start to finish until the target is found or the array ends. It works on ANY array — sorted or unsorted.

- Start at index 0
- Compare each element with the target
- If match found → return the index
- If end of array reached without a match → return -1

7.2 Linear Search — Java Code

```
public static int linearSearch(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) {
            return i;    // found! return index
        }
    }
    return -1;           // not found
}
```

7.3 Linear Search Trace — [5, 3, 8, 1, 9], target = 8

- i=0: arr[0]=5 ≠ 8 → continue
- i=1: arr[1]=3 ≠ 8 → continue
- i=2: arr[2]=8 = 8 → FOUND! return index 2

Worst case: Target is last element or not in array → check all n elements

7.4 Linear Search — Big O

Best Case	O(1) — target is the very first element
Average Case	O(n) — target is somewhere in the middle
Worst Case	O(n) — target is last, or not found at all
Space	O(1) — no extra memory needed
Requirement	Works on UNSORTED arrays — no pre-condition needed

---

7.5 Binary Search

Binary Search finds a target in a SORTED array by repeatedly HALVING the search space. It compares the target to the MIDDLE element and eliminates half the remaining array each step.

REQUIREMENT: Array MUST be sorted first!

- Set low=0, high=n-1
- Calculate mid = (low + high) / 2
- If arr[mid] == target → FOUND, return mid
- If arr[mid] < target → target is in RIGHT half → set low = mid + 1
- If arr[mid] > target → target is in LEFT half → set high = mid - 1
- Repeat until low > high (not found → return -1)

7.6 Binary Search — Java Code

```
public static int binarySearch(int[] arr, int target) {
    int low = 0;
    int high = arr.length - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == target) {
            return mid;              // FOUND
        } else if (arr[mid] < target) {
            low = mid + 1;           // go RIGHT
        } else {
            high = mid - 1;          // go LEFT
        }
    }
    return -1;                       // not found
}
```

7.7 Binary Search Trace — [1, 3, 5, 7, 9, 11, 13], target = 7

Step 1: low=0, high=6 → mid=3 → arr[3]=7 == 7 → FOUND at index 3! ✓

Another example — target = 11:
Step 1: low=0, high=6 → mid=3 → arr[3]=7 < 11 → go RIGHT → low=4
Step 2: low=4, high=6 → mid=5 → arr[5]=11 == 11 → FOUND at index 5! ✓

Another example — target = 6 (not in array):
Step 1: low=0, high=6 → mid=3 → arr[3]=7 > 6 → go LEFT → high=2
Step 2: low=0, high=2 → mid=1 → arr[1]=3 < 6 → go RIGHT → low=2
Step 3: low=2, high=2 → mid=2 → arr[2]=5 < 6 → go RIGHT → low=3
Step 4: low=3 > high=2 → STOP → return -1 (not found)

7.8 Binary Search — Big O

Best Case	O(1) — target is the middle element on first check
Average Case	O(log n) — halves the search space each step

Worst Case	O(log n) — target at the very end or not found
Space	O(1) iterative | O(log n) recursive (call stack)
Requirement	Array MUST be sorted — cannot use on unsorted data

7.9 Linear Search vs Binary Search — Comparison

Property	Linear Search	Binary Search
Best Case	O(1)	O(1)
Worst Case	O(n) ← SLOWER	O(log n) ← FASTER
Sorted Required?	NO ✓ any array	YES ✗ must sort first
100 elements	Up to 100 checks	Up to 7 checks
1 million items	Up to 1,000,000	Up to 20 checks!
Best used for	Small/unsorted	Large sorted arrays

---

8. Quick Reference Summary

Topic	Key Facts
Big O — O(1)	Constant time. Array index access. Best.
Big O — O(log n)	Logarithmic. Binary Search. Halves problem each step.
Big O — O(n)	Linear. One loop over n items.
Big O — O(n log n)	QuickSort avg, HeapSort, MergeSort. Best comparison sort.
Big O — O(n²)	Nested loops, Bubble Sort. Avoid for large n.
QuickSort	Divide & conquer. Pivot splits array. O(n log n) avg. O(n²) worst.
HeapSort	Uses Max-Heap. Always O(n log n). In-place: O(1) space.
Stack	LIFO. push/pop from TOP. Recursion uses call stack.
Queue	FIFO. enqueue at REAR, dequeue from FRONT.
Circular Queue	Wraps using rear=(rear+1)%CAPACITY. Solves wasted space.
Priority Queue	Min-Heap. Smallest element always dequeued first. O(log n).
BST	left < node < right. Search: O(log n) avg. In-Order = sorted.
In-Order (L→R→R)	D,B,A,C,E — gives sorted output for BST
Pre-Order (R→L→R)	A,B,D,C,E — use for copying/serializing tree
Post-Order (L→R→R)	D,B,E,C,A — use for deletion (children deleted first)
BFS / Level-Order	A,B,C,D,E — uses Queue, visits level by level
Insertion Sort	Pick key, shift larger elements right, insert. O(n) best, O(n²) worst. Stable.
Selection Sort	Find min, swap to front each pass. Always O(n²). NOT stable. Fewer swaps.
Linear Search	Check one by one. O(n) worst. Works on UNSORTED arrays.
Binary Search	Halve search space each step. O(log n). SORTED array required!

Good luck at 14:00!